

Incident Reporting System

AST Project Report

Muhammad Sohail (7178621)

July 9, 2026

<https://github.com/msohailse/incident-reporting-system>

Build command: `mvn clean verify`

Contents

1. Project Overview	3
2. Build Configuration: pom.xml	3
2.1. Dependencies	3
2.2. Testing Infrastructure	3
2.3. Test Execution Lifecycle	3
2.4. SonarCloud Integration	4
2.5. Profiles	4
3. Testing Strategy	4
3.1. Model Layer: Unit Tests	5
3.2. Controller Layer: Unit Tests	5
3.3. View Layer: UI Tests (AssertJ Swing)	5
3.4. Repository Layer: Integration Tests	5
3.5. Full Stack: End-to-End Test	6
3.6. Mutation Testing	6
3.7. Summary	7
4. Continuous Integration	7
5. Project Structure	7
5.1. Domain Model	7
5.2. Persistence Layer	8
5.2.1. JPA and Hibernate	8
5.2.2. Transaction Handling	8
5.2.3. One persistence.xml, Overridden in Tests	9
5.2.4. Docker and Automatic Database Start	9
5.3. MVC Architecture and Swing UI	10
5.3.1. Window Management and Panel Structure	10
5.3.2. Event Handling and Validation	10
6. Design Choices and Problems Solved	10
6.1. Dropping docker-maven-plugin	10
6.2. TDD in Practice: Three Red-Green-Refactor Cycles	10
6.3. Deciding on the Transaction Pattern (Book Section 8.7.1)	12
7. What I Found Interesting	12

1. Project Overview

The Incident Reporting System is a Java desktop application, built with Swing, where users can register, log in, and report incidents. Each incident has a severity level and can be tagged (a category label such as "fire" or "theft"). Data is persisted in a PostgreSQL database via JPA/Hibernate, with transactions implemented following the book's guidance: transaction boundaries are managed in a dedicated layer, never inside the repository. Docker is used to provision PostgreSQL for both integration tests and end-to-end tests, and is started automatically as part of the Maven build, both locally and in CI, without any manual setup step. The whole project was built following Test-Driven Development.

The domain has three main entities:

- **User:** registers and logs in to report incidents
- **Tag:** a category label like "fire" or "theft" describing what kind of incident it is
- **Incident:** the actual report, linked to whoever filed it (@ManyToOne User) and categorised by a single tag (@ManyToOne Tag)

2. Build Configuration: pom.xml

The project is a single Maven module: source and target are set to 1.8, targeting Java 8 language features and bytecode. Production dependencies are Hibernate, the PostgreSQL driver, and Picocli; test dependencies are JUnit, AssertJ Swing, Mockito, and Testcontainers. Test sources split across `test/`, `it/`, and `e2e/`, built and run separately by Surefire and Failsafe, with JaCoCo, PIT, and Coveralls wired in behind opt-in profiles.

2.1. Dependencies

Hibernate as the JPA provider and the `postgresql` JDBC driver, both in the default compile scope, plus Picocli, which parses the `--db-host/--db-port` style arguments `IncidentReportingSwingApp` accepts so the E2E test can launch the real application against a Testcontainers database.

2.2. Testing Infrastructure

Test-scope dependencies: JUnit 4.13.2 as the test runner, `assertj-swing-junit` to drive the Swing UI in tests, Mockito to mock the transaction manager/repository/view in controller tests, and the Testcontainers PostgreSQL module for the IT and E2E tests. Since Maven only looks for tests in `src/test/java` by default, `build-helper-maven-plugin` registers two extra test source directories: `/it/` and `/e2e/`.

2.3. Test Execution Lifecycle

The build lifecycle strictly separates unit testing from database-backed testing to ensure fast feedback loops and clean test environments. In Maven, lifecycle phases represent sequential stages in the build process; an execution configuration binds a specific plugin goal to a lifecycle phase, so the task is fulfilled when Maven reaches that stage. The execution follows a precise chronological flow:

1. **Source Registration:** During the `generate-test-sources` phase, `build-helper-maven-plugin` triggers its `add-test-source` goal, appending `/it/` and `/e2e/` to the project's test source roots.

2. **Unit and UI View Testing:** During the test phase, `maven-surefire-plugin` executes once, excluding any classes ending in `IT.java` or `E2E.java`. This is a single execution, so isolated unit tests and the AssertJ Swing view test run together in the same pass.
3. **Integration and End-to-End Testing:** `maven-failsafe-plugin` takes over during the `integration-test` and `verify` phases, with two separate executions: `it-tests` (includes `**/*IT.java`) and `e2e-tests` (includes `**/*E2E.java`). Each test class provisions its own temporary PostgreSQL container via `Testcontainers`.
4. **Coverage Reporting:** The `jacoco-maven-plugin` attaches its agents early in the build (`prepare-agent` and `prepare-agent-integration`). During the `post-integration-test` phase, it executes the `report` and `report-integration` goals to generate the XML coverage reports.

2.4. SonarCloud Integration

While the build lifecycle generates test results and coverage metrics, the SonarCloud scanner requires explicit configuration to locate these artifacts. Because the project utilizes custom test directories and multiple execution plugins, the default SonarCloud paths are insufficient. The properties block defines three crucial configurations to bridge this gap:

- **Test Directories:** The `sonar.tests` property explicitly registers the unit, integration, and end to end source directories so the scanner analyzes all test suites.
- **Execution Reports:** The `sonar.junit.reportPaths` property directs the scanner to the output directories of both Surefire and Failsafe to accurately reflect test passes and failures.
- **Coverage Metrics:** The `sonar.coverage.jacoco.xmlReportPaths` property provides the exact paths to both the unit and integration JaCoCo XML files, ensuring SonarCloud displays a unified coverage percentage.

2.5. Profiles

Three Maven profiles isolate tasks that shouldn't run on every build:

- **jacoco:** active by default (its activation condition is `skipCoverage != true`), so coverage is collected unless a developer passes `-DskipCoverage=true`. It excludes the domain models, `IncidentReportingSwingApp`, and `IncidentReportingSwingView` from the report, so coverage only reflects tested business logic.
- **mutation-testing:** has no activation block, so it never runs unless explicitly requested with `-Pmutation-testing`. `pitest-maven` targets the controller package only, via the `mutationCoverage` goal in the `verify` phase, with a `mutationThreshold` of 100 (the build fails if any mutant survives).
- **coveralls:** also requires an explicit `-Pcoveralls` flag. It binds `coveralls-maven-plugin` to the `verify` phase, uploading both the unit and integration JaCoCo reports (`jacoco.xml` and `jacoco-it.xml`) using a repo token read from the `COVERALLS_REPO_TOKEN` environment variable, never committed to the pom.

3. Testing Strategy

Tests are organised by architectural layer, not by tool. Each layer is tested at the tier where it needs exactly one real dependency and nothing more: models need nothing real, controllers mock everything below them, the view needs a real window but mocked controllers, repositories need a real database but nothing else, and the single end-to-end test needs everything real at once.

3.1. Model Layer: Unit Tests

Pure validation, no mocks needed.

- **UserTest**: name whitespace rejection, email format, password strength
- **TagTest**: title/description validation and whitespace normalisation
- **IncidentTest**: field validation, null checks, defaults, whitespace normalisation
- **StringNormalizerTest**: the shared whitespace helper, tested once at its source

3.2. Controller Layer: Unit Tests

The transaction manager, repository, and view are all Mockito mocks, so a controller test never touches a database or opens a window. `transactionManager` is stubbed once in `@Before` to execute the lambda it receives against the mock repository, matching the book's approach (p.264):

```
when(transactionManager.doInTransaction(any()))
    .thenAnswer(answer(
        (TransactionCode<?> code) ->
            code.apply(incidentReportingRepository)));
```

A typical test asserts that `findUserByEmail` returning null triggers `view.showError`, and that `doInTransaction` was called exactly once — proving both the business logic and the one-transaction-per-use-case rule together.

- **UserControllerTest**: login (not found, wrong password, success), register (duplicate email, success)
- **IncidentControllerTest**: reusing vs. creating a tag, correct fields saved, `view.incidentAdded` on success, one transaction even when a tag is created first

3.3. View Layer: UI Tests (AssertJ Swing)

IncidentReportingSwingViewTest drives the real `IncidentReportingSwingView` JFrame with mocked controllers underneath it. It runs under `@RunWith(GUITestRunner.class)`, extending `AssertJSwingJUnitTestCase`, and interacts with the window through a `FrameFixture/Robot`. Coverage spans initial panel state, button enable/disable while typing, error-label text, button clicks delegating to the right controller method, and the view reacting correctly to `userLoggedIn/showAllIncidents/incidentAdded` callbacks — no database involved. SonarCloud's "tests should include assertions" rule (S2699) is suppressed for this file, since it doesn't recognise AssertJ Swing fixture calls as assertions.

3.4. Repository Layer: Integration Tests

ITs run against a real PostgreSQL container started automatically by Testcontainers. Each class builds one `EntityManagerFactory` per class and clears all rows before every test so tests don't interfere with each other.

- **UserPersistenceIT**: save, find, merge (attached and detached), `findByEmail` match vs. no match, duplicate email constraint
- **TagPersistenceIT**: save, find, `findByTitle` match vs. no match, whitespace normalisation on save
- **IncidentPersistenceIT**: save, find, tag relationship, `isClosed` default, non-existent id
- **JpaTransactionManagerIT**: commit, rollback on exception, rollback on commit failure, plus repository coverage run through a real transaction

3.5. Full Stack: End-to-End Test

IncidentReportingSwingAppE2E is the only test with nothing mocked: a real `PostgreSQLContainer`, a seeded `User`, and the real `IncidentReportingSwingApp` entry point launched via `AssertJ Swing's ApplicationLauncher` against that container. It logs in, submits an incident through the real form, and asserts it appears in the real list. Since the submit button now runs on a background thread (Section 5.3.), the test polls with `Pause/Condition` rather than asserting immediately.

3.6. Mutation Testing

PIT checks that the tests above actually detect defects, not just that lines execute. It runs on demand only (`mvn clean verify -Pmutation-testing`), never in CI, scoped to `com.msohailse.app.incident.controller.*` with `mutationThreshold=100` – the build fails if a single mutant survives. The scoping decision is discussed in Section 6.2.. An earlier, unscoped run surfaced survivors in `UserController` and three `getId()` methods, which drove the fixes described in Section 6.2.; the current scoped run kills every generated mutant, shown in Figure 1:

```
>> Generated 18 mutations Killed 18 (100%)
>> Mutations with no coverage 0. Test strength 100%
>> Ran 39 tests (2.17 tests per mutation)
BUILD SUCCESS
```

```
=====
- Statistics
=====
>> Line Coverage (for mutated classes only): 43/43 (100%)
>> Generated 18 mutations Killed 18 (100%)
>> Mutations with no coverage 0. Test strength 100%
>> Ran 39 tests (2.17 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.423 s
[INFO] Finished at: 2026-07-09T02:37:30+02:00
[INFO] -----
```

Figure 1: The current PIT mutation-testing report for this project, showing 0 surviving mutants (18/18 killed) in the scoped controller package.

3.7. Summary

Test class	Tests	Type
UserTest	20	Unit
TagTest	13	Unit
IncidentTest	26	Unit
StringNormalizerTest	10	Unit
UserControllerTest	9	Unit (Mockito)
IncidentControllerTest	7	Unit (Mockito)
IncidentReportingSwingViewTest	18	UI (AssertJ Swing)
UserPersistenceIT	7	Integration
TagPersistenceIT	8	Integration
IncidentPersistenceIT	6	Integration
JpaTransactionManagerIT	10	Integration
IncidentReportingSwingAppE2E	1	End-to-end
Total	135	

Table 1: Test counts by class

Thus satisfying the required testing pyramid (unit > integration > end-to-end).

4. Continuous Integration

GitHub Actions runs the full build (`.github/workflows/maven.yml`) on every push and pull request, on `ubuntu-latest` with JDK 17 (Temurin), under `xvfb-run` so AssertJ Swing's UI tests can open real windows on a headless runner. `$SONARGOAL` is set to `sonar:sonar` only when the `SONAR_TOKEN` secret is present, so SonarCloud and Coveralls both run on every CI build; the rest of the pipeline (Surefire/Failsafe/JaCoCo/PIT execution) is the same lifecycle described in Section 2.3.. After the build, the Surefire report is uploaded as a build artifact so failed tests can be inspected without digging through raw logs. The main README file contains the required dynamic status badges for the build status, Coveralls code coverage, and SonarCloud quality metrics.

5. Project Structure

The project follows the standard MVC layers used throughout the course, under package `com.msosailse.app.incident:` `model`, `repository` (`.postgres` implementation), `controller`, and `view` (`.swing` implementation).

5.1. Domain Model

The domain consists of three entities: `User`, `Tag`, and `Incident`. To avoid a known SQL runtime error where PostgreSQL treats “user” as a reserved keyword, the user entity explicitly maps to the `users` table via `@Table(name="users")`.

Rather than duplicating validation logic, both `Tag` and `Incident` delegate whitespace normalisation (such as collapsing consecutive spaces in titles) to a shared `StringNormalizer` component. This component processes the string in a single pass using a `StringBuilder`, avoiding the overhead of regular expressions.

Finally, relationships are managed natively via JPA. Crucially, each incident has exactly one tag. The foreign key `tag_id` lives directly on the `incidents` table, mapping a `@ManyToOne` relationship where the same `Tag` row can be referenced by multiple incidents. A similar `@ManyToOne` relationship links the incident to the reporting `User`.

5.2. Persistence Layer

5.2.1. JPA and Hibernate

All three entities are annotated with `@Entity` and declare their table names explicitly. Hibernate reads these annotations at startup, creates the schema if it does not exist, and generates the `INSERT/SELECT` SQL for every repository call; none of it is written by hand.

5.2.2. Transaction Handling

I split the pattern into an abstract side and a concrete side, so the two responsibilities never leak into each other.

The abstract side:

- `IncidentReportingRepository`: a plain interface (`save/find*` for `User`, `Tag`, `Incident`), with no knowledge of JPA.
- `TransactionCode<T>`: a functional interface, `Function<IncidentReportingRepository, T>`. Controllers only ever see this interface type when they write their lambda.
- `TransactionManager`: interface, `<T> T doInTransaction(TransactionCode<T> code)`.

Because `TransactionCode<T>` is typed against the interface, not a concrete class, a controller's lambda (`repo -> repo.save(user)`) can never call anything Postgres-specific: the abstraction is enforced at compile time, not by convention.

The concrete implementation. Someone still has to build the real Postgres-backed repository, and per the book's rule it must happen inside the transaction, sharing the exact `EntityManager` the transaction was opened on. That is the one job of `JpaTransactionManager`, the only class in the codebase that calls `em.getTransaction()`:

```
public <T> T doInTransaction(TransactionCode<T> code) {
    EntityManager em = emf.createEntityManager();
    try {
        em.getTransaction().begin();
        T result = code.apply(new IncidentReportingPostgresRepository(em));
        em.getTransaction().commit();
        return result;
    } catch (RuntimeException e) {
        if (em.getTransaction().isActive()) {
            em.getTransaction().rollback();
        }
        throw e;
    } finally {
        em.close();
    }
}
```

This cannot be restructured to build the repository earlier or outside this method: the `EntityManager` does not exist until `createEntityManager()` runs here, and reusing one from a previous call would

break atomicity between unrelated transactions. So the concrete construction is deliberately confined to this single method, keeping the rest of the application, controllers and their tests, talking only to the abstraction.

Granularity. Controllers hold a `TransactionManager`, never an `EntityManager` or repository directly, and call `transactionManager.doInTransaction(repo -> { ... })` once per use case. `IncidentController.reportIncident` needs two writes in the same use case (find or create a `Tag` by title, then save the `Incident` referencing it), and both happen inside the same lambda against the same repository instance, so they commit or roll back together as one atomic transaction, matching the book's guidance (page 266) that transaction boundaries should align with use cases, not individual repository calls.

Testability. Because the abstraction boundary sits exactly at `IncidentReportingRepository`, controller unit tests stub `doInTransaction` with a Mockito `Answer` that invokes the lambda against a mock `IncidentReportingRepository`: real business logic gets exercised and verified without ever mocking `EntityManager`, a third-party type the book warns against mocking, and without touching a real database.

Verification. `JpaTransactionManager` itself is covered by `JpaTransactionManagerIT`, run against a real PostgreSQL `Testcontainers`: commit persists correctly, an exception thrown inside the lambda rolls back and leaves no partial state, and a duplicate-key constraint violation also triggers rollback rather than a half-committed write.

5.2.3. One persistence.xml, Overridden in Tests

There is a single persistence unit named `incident_reporting` in `src/main/resources/META-INF/persistence.xml` with default connection properties pointing at the local Docker container (`localhost:5432/incident_db`) for manual runs of the app. No separate test `persistence.xml` exists. Instead, `Persistence.createEntityManagerFactory(properties)` accepts a `Map<String, String>` of overrides as its second argument: any property in that map replaces the corresponding one from `persistence.xml` at runtime, without needing a second XML file.

Integration tests use this through a small shared helper, `PostgresITSupport.createEntityManagerFactory(PostgresITSupport)` which builds that override map from the running `Testcontainers` instance:

```
Map<String, String> properties = new HashMap<>();
properties.put("javax.persistence.jdbc.url", postgres.getJdbcUrl());
properties.put("javax.persistence.jdbc.user", postgres.getUsername());
properties.put("javax.persistence.jdbc.password", postgres.getPassword());
return Persistence.createEntityManagerFactory("incident_reporting",
    properties);
```

`Testcontainers` assigns PostgreSQL a random host port per test run, so the URL cannot be hardcoded anywhere; it has to be read from `postgres.getJdbcUrl()` after the container starts and injected at `EntityManagerFactory` creation time. This keeps one canonical entity-mapping definition (the `<class>` list, dialect, `hbm2ddl.auto`) shared by both production code and tests, so schema and mapping changes only need to happen in one place, while the connection target differs.

5.2.4. Docker and Automatic Database Start

Each of the four IT classes and the E2E class declares its own `Testcontainers @ClassRule`, a `PostgreSQLContainer<>("postgres:15")`, so the database starts automatically wherever the tests run, with no manual Docker command needed beyond a running daemon.

`IncidentReportingPostgresRepository` implements `IncidentReportingRepository` by delegating to three per-entity classes (`UserPostgresRepository`, `TagPostgresRepository`, `IncidentPostgresRepository`) sharing the same `EntityManager`; the interface itself is described above under Transaction Handling.

5.3. MVC Architecture and Swing UI

The UI follows MVC: the controller talks to the presentation layer only through the `IncidentReportingView` interface (`showError`, `showAllIncidents`, `userLoggedIn`, `incidentAdded`), never touching Swing components directly.

5.3.1. Window Management and Panel Structure

The application runs within a single `JFrame` to avoid popup clutter. A `CardLayout` manages four distinct screens (`LOGIN`, `REGISTER`, `MAIN`, and `INCIDENT`). Each screen is built once at startup using `GridBagLayout` and assigned a card name. This allows seamless navigation (e.g. `cardLayout.show(rootPanel, "MAIN")`) without rebuilding component references.

5.3.2. Event Handling and Validation

- **Dynamic Button State:** Action buttons are disabled by default. A shared `KeyAdapter` is attached to input fields, hooking into `keyReleased` to evaluate inputs and enable the button only when all required fields are populated.
- **Action Listeners:** Button click events are wired inline using lambda expressions. These listeners handle immediate UI resets, such as clearing error labels, and pass the trimmed inputs directly to the controller, keeping UI logic minimal.

6. Design Choices and Problems Solved

6.1. Dropping docker-maven-plugin

An earlier build used `docker-maven-plugin` (fabric8) to start a fixed-port PostgreSQL container alongside the per-class `Testcontainers` containers already used by every IT/E2E test; once nothing referenced that fixed port, it was removed as redundant. `docker-compose.yml` remains, only for running the app manually. That container is configured with `POSTGRES_HOST_AUTH_METHOD=trust`, so it does not require a password at all, keeping the repository free of any plaintext credential; IT and E2E tests are unaffected, since they authenticate against their own `Testcontainers`-managed PostgreSQL instances.

6.2. TDD in Practice: Three Red-Green-Refactor Cycles

Threading the submit button off the EDT. Red. Threading `submitIncidentButton`'s `reportIncident` call off the EDT (book Chapter 13.5, Multithreading) broke `IncidentReportingSwingViewTest`'s existing assertion. Before, the call was synchronous:

```
window.button("submitIncidentButton").click();
verify(incidentController).reportIncident("Server down", "DB offline",
    Severity.LOW, "infra", user);
```

Once the controller call moved to a background thread, this went red: `verify()` ran immediately after `.click()` returned, before the background thread had actually invoked the controller.

Green. The fix was test-side, not implementation-side: Mockito's timeout-based verify, which polls instead of asserting instantly, turned it green again:

```
verify(incidentController, timeout(1000)).reportIncident("Server down", "DB  
offline", Severity.HIGH, "infra", user);
```

Refactor. An earlier attempt (reverted, over-scoped versus the book's own single-button example) had threaded all three buttons and introduced a deferred panel-switch race. Refactoring back down to just `submitIncidentButton` matched the book's scope and fixed it.

Moving validation out of Incident's and Tag's setters. Red. Incident and Tag originally had a no-arg constructor plus public setters (`setTitle/setDescription` on Incident, `setTagTitle/setTagDescription` on Tag) that each independently called validation and `trimAllSpaces` before assigning the field. `IncidentController.reportIncident` built incidents this way:

```
Incident incident = new Incident();  
incident.setTitle(title);  
incident.setDescription(description);
```

This left a window where the object existed with an unvalidated title, right after `new Incident()` and before `setTitle()` ran, and if `setTitle` threw on an empty value, the object was left permanently in that half-built state, since every setter after the throw never reached. Running `mvn org.pitest:pitest-maven:mutationCoverage` against this design kept surfacing surviving mutants around the setter-validation logic: PIT could flip the `isNullOrEmpty` boundary check or remove the `trimAllSpaces` call, and the tests still passed, because nothing forced every construction path through a setter in a way that would catch a broken branch.

Green. The fix (commit 79e39e9) removed `setTitle/setDescription` from Incident and `setTagTitle` from Tag, moving validation and normalisation into the constructor itself through a single `validateAndNormalize` helper:

```
this.title = validateAndNormalize(title, "incidentTitle");  
this.description = validateAndNormalize(description, "incidentDescription")  
;
```

With the setters gone, title is set exactly once, in the constructor, and never touched again (Incident still exposes `setDescription` today, since a description is legitimately editable after creation; only title became immutable). There is exactly one code path that can produce a title, so any mutant PIT introduces into `validateAndNormalize` is exercised by the same constructor tests every time, with no separate untested setter branch left for it to hide in.

Refactor. This forced an immediate follow-up (commit 49838d9): `IncidentController.reportIncident` no longer compiled, since it still called the removed setters. It was rewritten to call the full constructor directly, `new Incident(title, description, severity, loggedInUser, tag)`, and `IncidentTest`, `IncidentControllerTest`, and three integration test classes were updated to match. The duplicated `isNullOrEmpty/trimAllSpaces` logic between Incident and Tag was later extracted into `StringNormalizer` (commit e3cbd40), with its own dedicated test class. Today PIT's scope in `pom.xml` is narrowed to `com.msoshaile.app.incident.controller.*` only, with `mutationThreshold=100` (the build fails if any mutant survives within that scope); model classes like Incident are no longer directly targeted by mutation testing, but this fix is what made the constructor-based design possible to reason about safely in the first place (see Figure 1, Section 3.).

Password validation on User. Red. `setPassword(String)` originally just assigned the field; a failing test asserting a too-short password should throw forced the first check. **Green.** A minimal length

check was added, then a digit check and an uppercase check, each driven by its own failing test before the corresponding `if` was written, finishing with a positive case confirming a valid password is actually stored. **Refactor.** The same discipline was reused for `setEmail`, and one `validateNameString` helper replaced duplicated logic in `setFirstName/setLastName`.

6.3. Deciding on the Transaction Pattern (Book Section 8.7.1)

This design is based on the book’s reference pattern, but not copied from a working example: the book only shows the abstract `TransactionManager/TransactionCode` shape and a non-database illustration (`TransactionalEmployeeManager` in `mockito-example`).

7. What I Found Interesting

The transaction rollback behaviour needed careful thought beyond just undoing SQL: a failed `persist` marks the transaction “rollback only”, and without an explicit `rollback()` before the next operation, Hibernate throws on the next call regardless of what it is. Centralising this in `JpaTransactionManager` (Section 5.2.) keeps that rule in exactly one place.

My favourite technique in the project is how the controller tests handle the transaction layer: the mocked `TransactionManager` is stubbed once with a Mockito answer that runs the controller’s lambda directly against a mock repository (Section 3.). The controller has no idea the transaction is fake, yet the test verifies both the business logic and that exactly one transaction is used per use case, with no database anywhere.